Estructuras de Datos

2026
Primer Cuatrimestre

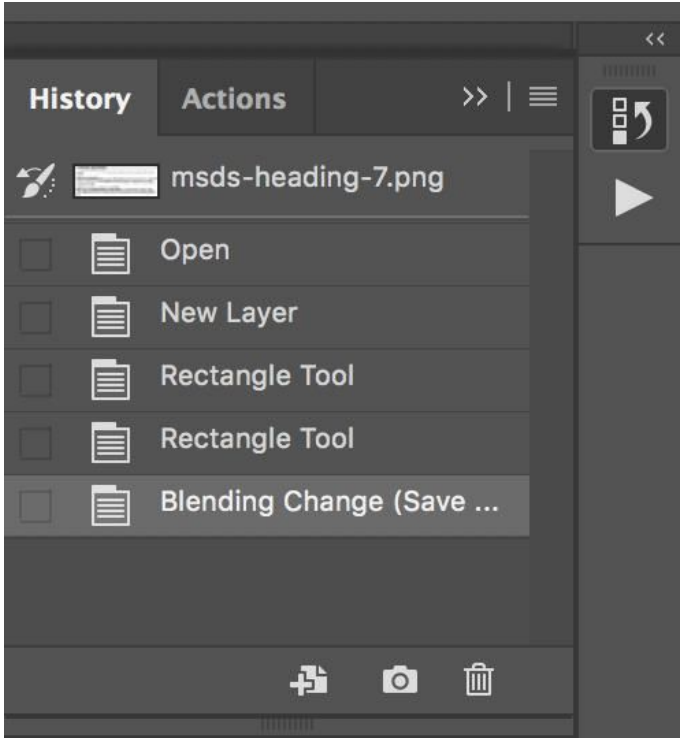
Repaso última clase

- Excepciones
  - Try-catch-finally
  - Checked/unchecked
- Implementación
  - TDA Pila con arreglo
  - TDA Cola con arreglo (circular)

Aplicaciones
TDA Pila / TDA Cola

Aplicaciones pila

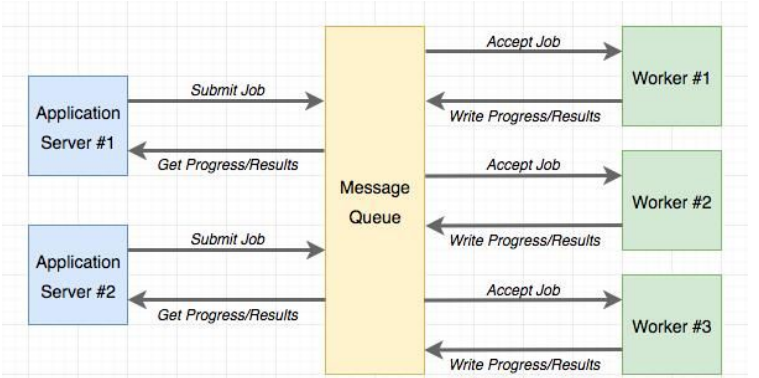
- Undo & redo (deshacer y rehacer)
- Navegación (botón hacia atrás)
- Llamado de funciones y recursividad
- Evaluación de expresiones y compiladores



Aplicaciones cola

- Sistemas operativos: procesos esperando por recursos
- Sistemas de impresión: varios procesos que envían a imprimir
- Exploración de grafos: sistemas de recomendación que nos indican qué ver como siguiente ítem
- Sistemas de mensajería / buffering: datos por streaming





TDA Pila / ADT Stack

push(e): Inserta el elemento e en el tope de la pila

pop(): Elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, produce una situación de error.

isEmpty(): Retorna verdadero si la pila no contiene elementos y falso en caso contrario.

top(): Retorna el elemento del tope de la pila. Si se aplica a una pila vacía, produce una situación de error.

size(): Retorna un entero natural que indica cuántos elementos hay en la pila.

NUESTRO TDA Pila

```
1  package TDAPila;
2
3  /**
4   * Interface Stack
5   * @author Catedra de Estructuras de Datos, Departamento de Cs. e Ing. de la Computaci□n,
6   */
7
8  public interface Stack<E> {
9
10     /**
11      * Consulta la cantidad de elementos de la pila.
12      * @return Cantidad de elementos de la pila.
13      */
14     public int size();
15
16     /**
17      * Consulta si la pila est□ vac□a.
18      * @return Verdadero si la pila est□ vac□a, falso en caso contrario.
19      */
20     public boolean isEmpty();
21
22     /**
23      * Examina el elemento que se encuentra en el tope de la pila.
24      * @return Elemento que se encuentra en el tope de la pila.
25      * @throws EmptyStackException si la pila est□ vac□a
```

NUESTRO TDA Pila

```
18      * @return Verdadero si la pila est vac a, falso en caso contrario.
19      */
20     public boolean isEmpty();
21
22     /**
23      * Examina el elemento que se encuentra en el tope de la pila.
24      * @return Elemento que se encuentra en el tope de la pila.
25      * @throws EmptyStackException si la pila est vac a.
26      */
27     public E top();
28
29     /**
30      * Inserta un elemento en el tope de la pila.
31      * @param element Elemento a insertar.
32      */
33     public void push(E element);
34
35     /**
36      * Remueve el elemento que se encuentra en el tope de la pila.
37      * @return Elemento removido.
38      * @throws EmptyStackException si la pila est vac a.
39      */
40     public E pop();
41 }
```

TDA Cola / ADT Queue

enqueue(e): Inserta el elemento e en el **back/tail/rear/rabo** de la cola

dequeue(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía se produce un error.

isEmpty(): Retorna verdadero si la cola no contiene elementos y falso en caso contrario.

front(): Retorna el elemento del frente de la cola. Si la cola está vacía se produce un error.

size(): Retorna un entero natural que indica cuántos elementos hay en la pila.

NUESTRO TDA Cola

```
1  package TDACola;
2  /**
3   * Interface Queue
4   * @author Catedra de Estructuras de Datos, Departamento de Cs. e Ing. de la Computaci□n,
5   */
6
7  public interface Queue <E>
8  {
9      /**
10      * Devuelve la cantidad de elementos en la cola.
11      * @return Cantidad de elementos en la cola.
12      */
13     public int size();
14
15     /**
16      * Consulta si la cola est□ vac□a.
17      * @return Verdadero si la cola est□ vac□a, falso en caso contrario.
18      */
19     public boolean isEmpty();
20
21     /**
22      * Inspecciona el elemento que se encuentra en el frente de la cola.
23      * @return Elemento que se encuentra en el frente de la cola.
24      * @throws EmptyQueueException si la cola est□ vac□a.
25      */
```

NUESTRO TDA Cola

```
17      * @return Verdadero si la cola est vac a, falso en caso contrario.
18      */
19     public boolean isEmpty();
20
21     /**
22      * Inspecciona el elemento que se encuentra en el frente de la cola.
23      * @return Elemento que se encuentra en el frente de la cola.
24      * @throws EmptyQueueException si la cola est vac a.
25      */
26     public E front();
27
28     /**
29      * Inserta un elemento en el fondo de la cola.
30      * @param element Nuevo elemento a insertar.
31      */
32     public void enqueue(E element);
33
34     /**
35      * Remueve el elemento en el frente de la cola.
36      * @return Elemento removido.
37      * @throws EmptyQueueException si la cola est vac a.
38      */
39     public E dequeue();
40 }
```

Genericidad: combinando tipos

- Una cola de booleanos:
  - Queue<Boolean>
- Una pila de colas de caracteres:
  - Stack<Queue<Character>>
- Una cola de pilas de colas de enteros:
  - Queue<Stack<Queue<Integer>>>

Resolvamos problemas con los TDAs Pila y Cola

Problema 1

Dada una cola de pilas de caracteres, determinar cuántas veces aparece una letra c dada. Se pueden destruir los datos.

Problema 1

```
13    public static int contarLetra(Queue<Stack<Character>> colaDePilas, Character c) {
14        int resultado = 0;
15        while (!colaDePilas.isEmpty()) {
16            Stack<Character> pilaActual = colaDePilas.dequeue();
17            while (!pilaActual.isEmpty()) {
18                char actual = pilaActual.pop();
19                if (c == actual) {
20                    resultado++;
21                    }
22                }
23            }
24        return resultado;
25    }
```

Problema 1

```
13    public static int contarLetra(Queue<Stack<Character>> colaDePilas, Character c) {
14        int resultado = 0;
15        while (!colaDePilas.isEmpty()) {
16            Stack<Character> pilaActual = colaDePilas.dequeue();
17            while (!pilaActual.isEmpty()) {
18                char actual = pilaActual.pop();
19                if (c == actual) {
20                    resultado++;
21                    }
22                }
23            }
24        return resultado;
25    }
```

Iteración exhaustiva

Problema 2

Dada una cola de pilas de caracteres, determinar si aparece una letra c dada. Se pueden destruir los datos.

Problema 2

```
27  public static boolean contieneLetra(Queue<Stack<Character>> colaDePilas, char cLetra)
28      boolean encontre = false;
29      while (!colaDePilas.isEmpty() && !encontre) {
30          Stack<Character> pilaActual = colaDePilas.dequeue();
31          while (!pilaActual.isEmpty() && !encontre) {
32              char elemento = pilaActual.pop();
33              if (elemento == cLetra) {
34                  encontre = true;
35              }
36          }
37      }
38      return encontre;
39  }
```

Problema 2

```
27  public static boolean contieneLetra(Queue<Stack<Character>> colaDePilas, char cLetra)
28      boolean encontre = false;
29      while (!colaDePilas.isEmpty() && !encontre) {
30          Stack<Character> pilaActual = colaDePilas.dequeue();
31          while (!pilaActual.isEmpty() && !encontre) {
32              char elemento = pilaActual.pop();
33              if (elemento == cLetra) {
34                  encontre = true;
35              }
36          }
37      }
38      return encontre;
39  }
```

Iteración no exhaustiva

IMPORTANTE

En esta materia, resolver un problema con un recorrido exhaustivo cuando era posible hacerlo con un recorrido NO exhaustivo, es considerado **una mala decisión y es penalizado** (les va a restar puntos en el parcial/final)

IMPORTANTE 2

También debemos estar atentos a si nos piden una solución para un tipo de datos en particular o con genericidad para cualquier tipo de datos

Problema 3

Invertir una pila

Problema 3

Invertir una pila

```
41    public static <E> Stack<E> invertirPila(Stack<E> original) {
42        Stack<E> resultado = new MiPilaBasica<>();
43        while (!original.isEmpty()) {
44            E elemento = original.pop();
45            resultado.push(elemento);
46        }
47        return resultado;
48    }
```

Problema 4

Invertir una pila sin que se pierda el contenido original

Problema 4

Invertir una pila sin que se pierda el contenido original

```
50    public static <E> Stack<E> invertirPilaSinDestruir(Stack<E> original) {
51        Stack<E> copiaOriginal = new MiPilaBasica<>();
52        Stack<E> resultado = new MiPilaBasica<>();
53        // vaciar la pila original en dos auxiliares
54        while (!original.isEmpty()) {
55            E elemento = original.pop();
56            copiaOriginal.push(elemento);
57            resultado.push(elemento);
58        }
59        // restaurar la pila original
60        while (!copiaOriginal.isEmpty()) {
61            original.push(copiaOriginal.pop());
62        }
63        return resultado;
64    }
```

Problema 4 (bis)

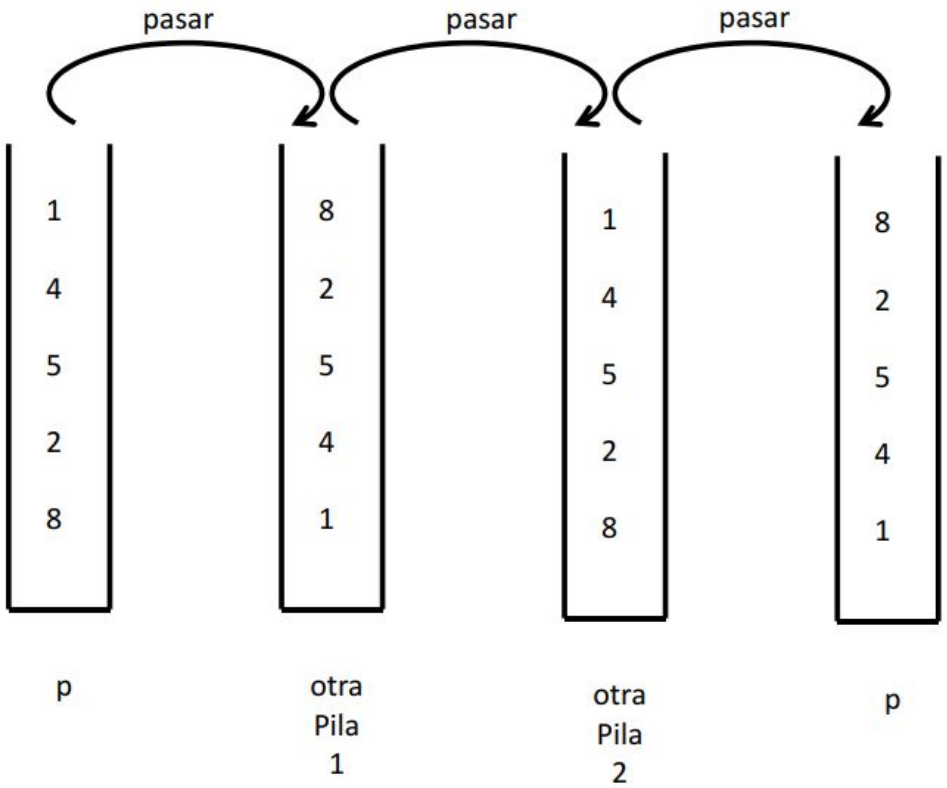
Invertir una pila sin que se pierda el contenido original. En lugar de devolver una nueva pila, la misma pila dada como parámetro deberá quedar invertida.

Qué pasa si hago:

```
41    public static <E> Stack<E> invertirPila(Stack<E> original) {
42        Stack<E> resultado = new MiPilaBasica<>();
43        while (!original.isEmpty()) {
44            E elemento = original.pop();
45            resultado.push(elemento);
46        }
47        return resultado;
48    }
```

original = resultado;

Estrategia


pasar
pasar
pasar
1
4
5
2
8
8
2
5
4
1
1
4
5
2
8
8
2
5
4
1
p
otra Pila 1
otra Pila 2
p

Problema 5

Dada una pila y un elemento, intercalar el elemento ingresado entre cada dos elementos de la pila. No se debe retornar una pila nueva; la pila modificada debe ser la misma pila que se ingresó.

Problema 5

Dada una pila y un elemento, intercalar el elemento ingresado entre cada dos elementos de la pila. No se debe generar una pila nueva, la pila modificada es la misma pila que se ingresó.

```
66    public static <E> void intercalar(Stack<E> pila, E elem) {
67        if (pila.size() <= 1) return;
68        E top = pila.pop();
69        intercalar(pila, elem);
70        pila.push(elem);
71        pila.push(top);
72    }
```

Problema 6

Chequear si un código java hace uso correcto de { y }

Problema 6

```
74    public static boolean llavesBalanceadas(String codigo) {
75        Stack<Character> pila = new MiPilaBasica<>();
76        int i = 0;
77        boolean bienFormado = true;
78        while (i < codigo.length() && bienFormado) {
79            char c = codigo.charAt(i);
80            if (c == '{') {
81                pila.push(c);
82            } else if (c == '}') {
83                if (pila.isEmpty()) {
84                    bienFormado = false; // Se cerró una llave sin haber abierto
85                } else {
86                    pila.pop();
87                }
88            }
89            i++;
90        }
91        return bienFormado && pila.isEmpty();
92    }
```

Problema 6 (bis)

Chequear si un código java hace uso correcto de los símbolos de agrupación en una expresión aritmética. Los símbolos de agrupación son:

• Paréntesis: (y)

• Llaves: { y }

• Corchetes: [y]

Cada símbolo de apertura debe coincidir su símbolo de cierre. Ejemplos:

1. Correcto: [(5 + x) – (y – z)]

2. Correcto: () (()) {[({})]}

3. Correcto: [[( ) (( ))]] { ( ) }

4. Incorrecto:) (()) []

5. Incorrecto: ([ ]

6. Incorrecto: (( ]

Evaluación de expresiones

Evaluaremos expresiones en notación polaca inversa (postfija)

Infija	Postfija (Polaca inversa)
3	3
3 + 4	3 4 +
7 * 5	7 5 *
3 + 4 * 5	3 4 5 * +
4 * 5 + 3	4 5 * 3 +
3 + 4 + 1	3 4 + 1 +
3 * 4 + 5 * 6	3 4 * 5 6 * +

Problema 7

Dada una cola con una expresión aritmética en notación infija

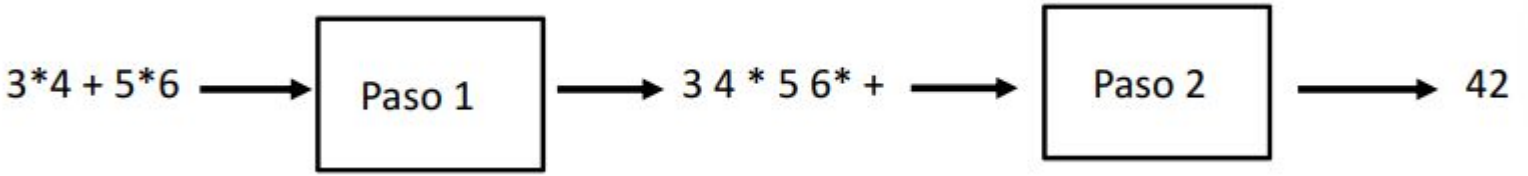
Paso 1: Pasar a la notación postfija.

Paso 2: Evaluar la expresión en notación postfija.

Evaluación de expresiones: esquema general

Solución:

- Entrada: Expresión aritmética infija válida con operadores +, -, *, / binarios, con * y / con mayor precedencia que + y - , sin paréntesis
- Paso 1: Traducir expresión infija en postfija (está en el apéndice).
- Paso 2: Evaluar expresión postfija

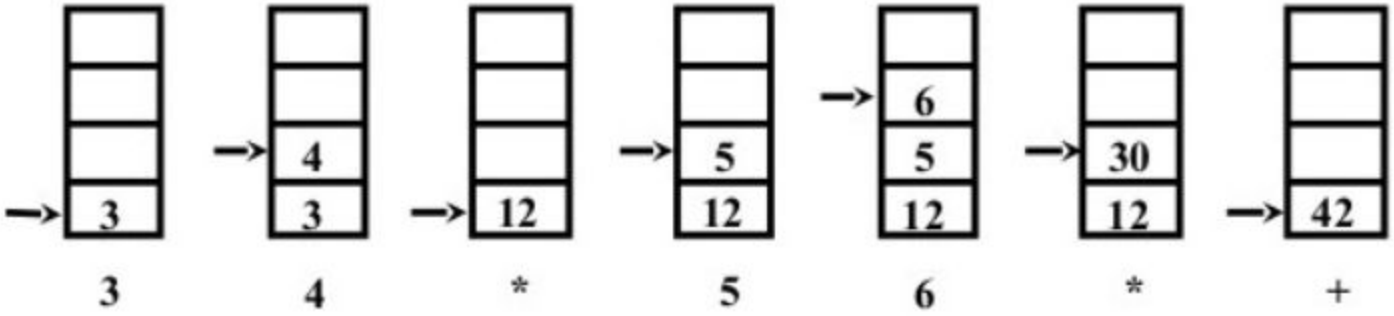


Paso 2: Evaluar expresiones en notación postfija

- Dada una expresión en notación postfija, se comienza con una pila vacía.
- Se consumen los símbolos de a uno por vez:
  - Los operandos se apilan.
  - Con cada operador se desapilan dos operandos, se aplica el operador y se vuelve a apilar el resultado.
- Si al finalizar la expresión:
  - la pila tiene un elemento, ese elemento es el resultado de la expresión.
  - la pila tiene más de un elemento, la expresión estaba incorrectamente formada.
- Si al tratar de aplicar un operador no hay por lo menos dos elementos en la pila, la expresión está mal formada.

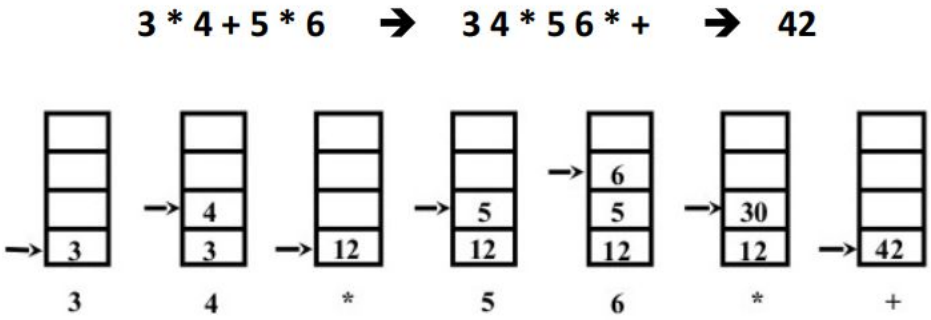
Paso 2: Evaluar expresiones en notación postfija

3 * 4 + 5 * 6 → 3 4 * 5 6 * + → 42



Paso 2: Pseudocódigo

```
P ← new Pila()
Repetir
        átomo ← E.dequeue()
        si átomo es operando entonces
                P.push( átomo )
        sino // átomo es un operador
        operador ←  átomo
        operando2 ← P.pop()
        operando1 ← P.pop()
        resultado ← operar( operador, operando1, operando2 )
        P.push( resultado )
Hasta E.empty()
Resultado  P.pop()
Retornar resultado
```



Problema 8

Dada una cola de pilas de elementos caracteres, retornar una pila de colas de caracteres. Los caracteres dentro de cada estructura deben quedar en el mismo orden.

THE END

Apéndice

Conversión de expresiones en notación infija a postija

- Este algoritmo se atribuye a Donald Knuth



Donald Knuth

78 idiomas

Artículo Discusión Leer Editar Ver historial Herramientas

Donald Ervin Knuth (Milwaukee, Wisconsin; 10 de enero de 1938) es un reconocido experto en ciencias de la computación estadounidense y matemático, famoso por su fructífera investigación dentro del análisis de algoritmos y compiladores.[1]

Es Profesor Emérito de la Universidad de Stanford.[2]

Biografía [editar]

Primeros años [editar]

Knuth nació en Milwaukee, Wisconsin, hijo de Ervin Henry Knuth y Louise Marie Bohning.[3] Describe su herencia como «alemana luterana del Medio Oeste».[4]:66 Su padre tenía una pequeña imprenta y enseñaba contabilidad.[5] Donald, estudiante del Milwaukee Lutheran High School, pensó en formas ingeniosas de resolver problemas. Por ejemplo, en octavo grado, participó en un concurso para encontrar el número de palabras que las letras en «Ziegler's Giant Bar» («Barra Gigante de Ziegler») podían ser reordenadas; los jueces habían identificado 2500 palabras de este tipo. Con el tiempo ganado fuera de la escuela debido a un fingido dolor de estómago, y trabajando el problema en sentido contrario, Knuth utilizó un diccionario no abreviado y determinó si cada entrada del diccionario podía formarse utilizando las letras de la frase. Usando este algoritmo, identificó más de 4500 palabras,

Donald Knuth

Donald Knuth en 2011

Información personal

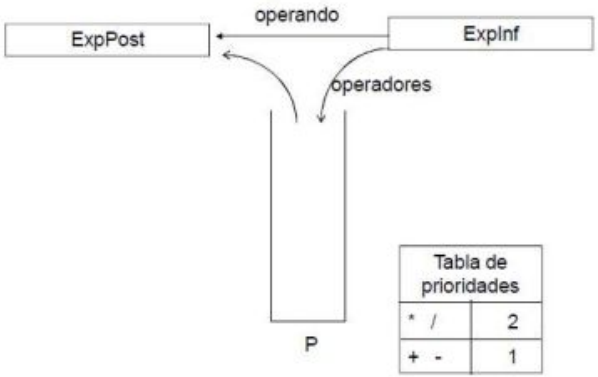
Nombre en inglés	Donald Ervin Knuth
Nacimiento	10 de enero de 1938 (88 años) Milwaukee, Wisconsin, Estados Unidos
Nacionalidad	Estadounidense

Paso 1: Paso de notación infija a postfija

Entrada: ExpInf es una cola que contiene una expresión válida en notación infija, sin parentizar y con operadores aditivos (+, -) y multiplicativos (*,/)

Salida: ExpPost es una cola que al finalizar queda con la expresión en notación postfija

- Se usa una pila auxiliar de operadores según el esquema que se detalla abajo
- Luego de retirar el último elemento de la cola ExpInf, se desapila todo lo que resta y se lo ingresa en la cola ExpPost



Paso 1: Paso de notación infija a postfija

```
ExpPost ← new Cola()
P ← new Pila()
Repetir
        átomo ← ExpInf.dequeue()
        si átomo es operando entonces
                ExpPost.enqueue( átomo )
        sino // átomo es un operador
                Mientras (no P.isEmpty()) Y (prioridad(átomo) <= prioridad(P.top()) )
                        ExpPost.enqueue( P.pop() )
                P.push( átomo )
Hasta ExpInf.isEmpty()
Mientras no P.isEmpty()
        ExpPost.enqueue( P.pop() )
```

3 * 4 + 5 * 6 ➔ 3 4 * 5 6 * +